

ISTQB® CTFL v4.0

Chapter 1

Fundamentals of Testing

Learning Objectives

1.1 What is Testing?

FL-1.1.1 (K1) Identify typical test objectives

FL-1.1.2 (K2) Differentiate testing from debugging

1.2 Why is Testing Necessary?

FL-1.2.1 (K2) Exemplify why testing is necessary

FL-1.2.2 (K1) Recall the relation between testing and quality assurance

FL-1.2.3 (K2) Distinguish between root cause, error, defect, and failure

1.3 Testing Principles

FL-1.3.1 (K2) Explain the seven testing principles

Learning Objectives cont.

1.4 Test Activities, Testware and Test Roles

FL-1.4.1 (K2) Summarize the different test activities and tasks

FL-1.4.2 (K2) Explain the impact of context on the test process

FL-1.4.3 (K2) Differentiate the testware that supports the test activities

FL-1.4.4 (K2) Explain the value of maintaining traceability

FL-1.4.5 (K2) Compare the different roles in testing

1.5 Essential Skills and Good Practices in Testing

FL-1.5.1 (K2) Give examples of the generic skills required for testing

FL-1.5.2 (K1) Recall the advantages of the whole team approach

FL-1.5.3 (K2) Distinguish the benefits and drawbacks of independence of testing

Chapter 1 exam focus and key terms

What students must recognize, differentiate, and explain.



Exam
focus



Exam emphasis

- 8 questions from Chapter 1
- K1 = 2 questions
- K2 = 6 questions
- No K3 calculations in this chapter



High-risk vocabulary

- testing $\neq$ debugging
- error $\neq$ defect $\neq$ failure
- QA $\neq$ QC
- verification $\neq$ validation



Question verbs

- identify
- recall
- differentiate
- summarize
- explain
- compare

Core keywords: coverage, debugging, defect, error, failure, quality, quality assurance, root cause, test basis, test case, test objective, testware, validation, verification.

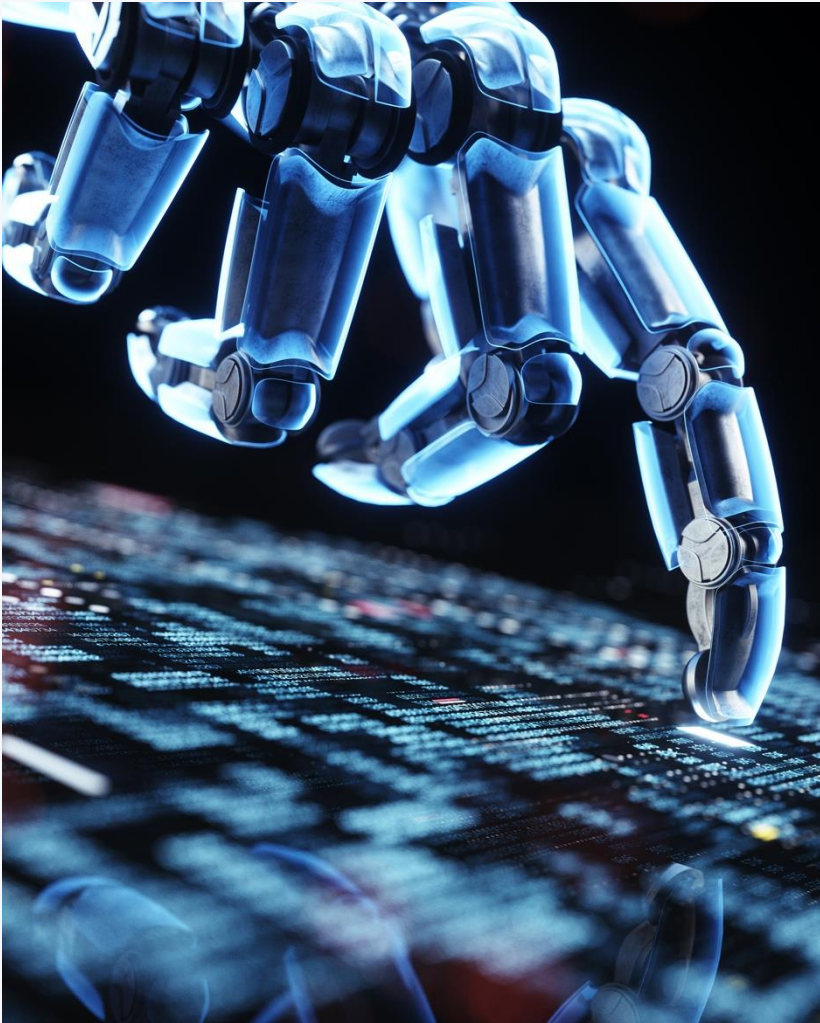


A customer is shopping on Shopee-like e-commerce app.
The shopping cart shows a total of **RM120**.
The customer applies a **10% discount voucher**.
At checkout, the system displays **RM120 instead of RM108**.

What is the failure?

What could be the defect?

What could be the root cause?



1.1 What is Testing?

1.1 What is Testing?

Software testing is a set of activities to discover defects and evaluate the quality of software artifacts.

These artifacts, when being tested, are known as test objects.

1.1 Common misconception

"It only consists of executing tests (i.e., running the software and checking the test results). "

However, software testing also **includes other activities** and must be aligned with the software development lifecycle

"Testing focuses entirely on verifying the test object. "

Whilst testing involves verification, i.e., checking whether the system meets specified requirements, it also involves validation, which means checking whether the system meets users' and other stakeholders' needs in its operational environment.



1.1 Verification vs validation

Both belong inside testing, but they answer different questions.



Verification

- Checks whether specified requirements have been fulfilled.
- Question: “Are we building the product right?”
- Looks at conformance to specification.



Validation

- Checks whether the product meets stakeholder needs in operation.
- Question: “Are we building the right product?”
- Looks at usefulness in the real context.

Both matter



a feature matches the spec perfectly but users still hate it—what did we miss?

1.1 Static testing vs dynamic testing

Testing can happen with or without executing the software.



Static testing

- No execution of the software.
- Examples: reviews, walkthroughs, static analysis of work products. (requirement, user stories, source code)
- Can find defects directly and early.



Dynamic testing

- Executes the software.
- Examples: component, integration, system, acceptance tests.
- Can trigger failures caused by defects.

Testing is not only a technical activity. It also needs to be properly planned, managed, estimated, monitored and controlled (see chapter 5).

Testers use tools (see chapter 6), but it is important to remember that testing is largely an intellectual activity, requiring the testers to have specialized knowledge, use analytical skills and apply critical thinking and systems thinking (Myers 2011, Roman 2018).

The ISO/IEC/IEEE 29119-1 standard provides further information about software testing concepts.

1.1.1 Test Objectives



1.1.1 Test objectives

Typical objectives are context dependent, but the syllabus lists a standard set.

Evaluating work products such as requirements, user stories, designs, and code

Triggering failures and finding defects

Ensuring required coverage of a test object

Reducing the level of risk of inadequate software quality

Verifying whether specified requirements have been fulfilled

Verifying that a test object complies with contractual, legal, and regulatory requirements

Providing information to stakeholders to allow them to make informed decisions

Building confidence in the quality of the test object

Validating whether the test object is complete and works as expected by the stakeholders

1.1.1 Objectives depend on context

Different test levels emphasize different objectives.



1.1.1

Objectives of testing can vary, **depending upon the context**, which includes

- the work product being tested
- the test level
- risks
- the software development lifecycle (SDLC) being followed
- factors related to the business context, e.g., corporate structure, competitive considerations, or time to market.



Component testing

- Often emphasizes triggering failures early.
- May also aim at increasing code coverage and fast feedback to developers.



Acceptance testing / UAT

- Emphasizes confidence, validation, and release information.
- Should not be the first place where many serious defects are discovered.

1.1.2 Testing & Debugging

1.1.2 Testing and debugging

Related, but not the same activity.



1.1.2



Testing

- Triggers failures or directly finds defects.
- Compares actual and expected results.
- Provides defect information.



Debugging

- Failure reproduction
- Diagnosis the cause / root cause.
- Fixes the cause in the work product.



1.1.2 Example: triangle overflow

A textbook example that cleanly separates testing from debugging.

- Tester runs triangle.exe with very large but valid values.
- Observed result = “no”; expected result = “yes”.
- Tester reports the failure.
- Developer diagnoses integer overflow and fixes the code.
- Tester reruns the test to confirm the fix.

Lesson:
Testing discovers information.
Debugging removes causes.
Confirmation testing verifies the fix.

1.1.2 Testing & Debugging (static testing)

When **static testing** identifies a defect, debugging is concerned with removing it.

There is no need for reproduction or diagnosis, since static testing directly finds defects, and cannot cause failures (see chapter 3).

1.1.2 Quick activity: classify the activity

Label each item as testing or debugging.



1.1.2

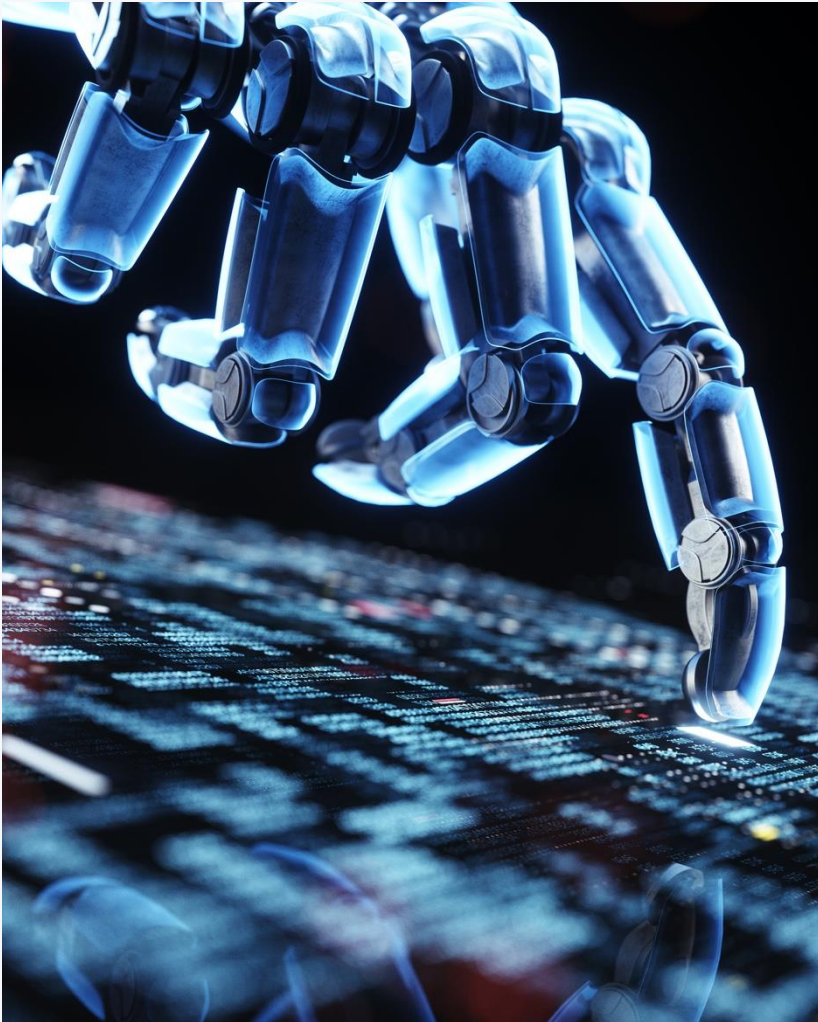
A. Execute the test and observe failure

B. Understand why the failure occurs

C. Change code to correct the cause

D. Retest after the fix

E. Review code to locate the defect



1.2 Why is Testing Necessary?



1.2 Why Is Testing Necessary?

Testing supports quality control and helps projects succeed within scope, time, and budget.



Testing helps because it ...

- finds defects in software and documentation
- reveals gaps or ambiguities in specifications
- reduces the risk of failures in operation



Testing may also be required for ...

- contracts
- legal obligations
- regulatory standards



Risk lens

For each anomaly in an application, ask:

- What is the probability?
- What is the impact?



Would you fix every defect you find? Why or why not?

1.2.1 Testing Contribution to Success

1.2.1 Testing's contribution to success

Testing contributes through product quality, process quality, project goals, and people skills.



1.2.1

Product quality

Detect defects, reduce risk, support release confidence

Process quality

Feedback shows where the development process needs improvement

Project goals

Supports scope, budget, schedule, and release decisions

People skills

Reviews and collaboration grow team capability

1.2.2 Testing and Quality Assurance (QA)

Testing and Quality Assurance (QA)

While people often use the terms “testing” and “quality assurance” (QA) interchangeably, testing and QA are not the same. Testing is a form of quality control (QC).

QC is a product-oriented, corrective approach that focuses on those activities supporting the achievement of appropriate levels of quality. Testing is a major form of quality control, while others include formal methods (model checking and proof of correctness), simulation and prototyping.

QA is a process-oriented, preventive approach that focuses on the implementation and improvement of processes. It works on the basis that if a good process is followed correctly, then it will generate a good product. QA applies to both the development and testing processes, and is the responsibility of everyone on a project.

Test results are used by QA and QC.

In QC they are used to fix defects, while in QA they provide feedback on how well the development and test processes are performing.

1.2.2 Testing and Quality Assurance (QA)

Testing is not the same as quality assurance.



1.2.2

Quality Management



Quality assurance (QA)

- Process-oriented and preventive approach
- Focuses on establishing, implementing, monitoring, improving and adhering to quality related processes
- Basis: if a good process is followed correctly, then it will generate a good product.
- Goal: confidence that quality requirements will be fulfilled



Quality control (QC) and testing

- Product-oriented and corrective / evaluative
- Testing is a major form of QC
- Uses results to identify and support correction of defects

Test results are used by QA and QC.

In QC they are used to fix defects, while in QA they provide feedback on how well the development and test processes are performing.



INTERACTIVE: Quick check: “writing a defect management process” fits QA or QC? What about executing system tests?

1.2.3 Errors, Defects, Failures and Root Causes

1.2.3 Error → defect → failure → root cause

Four terms candidates often confuse.



Important nuance

- A defect may exist without causing a failure in every execution.
- Some failures come from environmental conditions, not directly from human error.
- Root cause analysis aims to stop similar defects from recurring.



INTERACTIVE: Ask learners to create their own one-sentence chain using a banking, e-commerce, or mobile app example.

1.2.3 Quick check: beyond the chain

Not every unexpected test result proves a product defect.



False positive

- Test fails, but it should actually pass.
- Possible causes: results of errors related to test execution, defects in test data, test environment, other testware etc..



False negative

- Test passes, but it should actually fail.
- Possible causes: outdated expected result, weak oracle, masking effect.



Defect masking

- One defect hides another defect, or their effects cancel each other.
- Difficult to detect.



INTERACTIVE: Ask: if a test fails because the environment is broken, what exactly should you report?

1.2.3 Worked example: e-store discount problem

1.2.3



Map a real scenario to the four terms.

Scenario: A defect in the code causes incorrect calculation of the discount on bulk purchases in the e-store, resulting in customer complaints. The defective code was written based on a user story, but the product owner misunderstood the discount calculation rules and wrote the story wrong.

Consequence?

Failure?

Defect?

Root cause?

Error?



1.2.3 Worked example: e-store discount problem

Map a real scenario to the four terms.

Scenario: A defect in the code causes incorrect calculation of the discount on bulk purchases in the e-store, resulting in customer complaints. The defective code was written based on a user story, but the product owner misunderstood the discount calculation rules and wrote the story wrong.

Consequence

Customer complaints

Failure

Incorrect discount result
shown

Defect

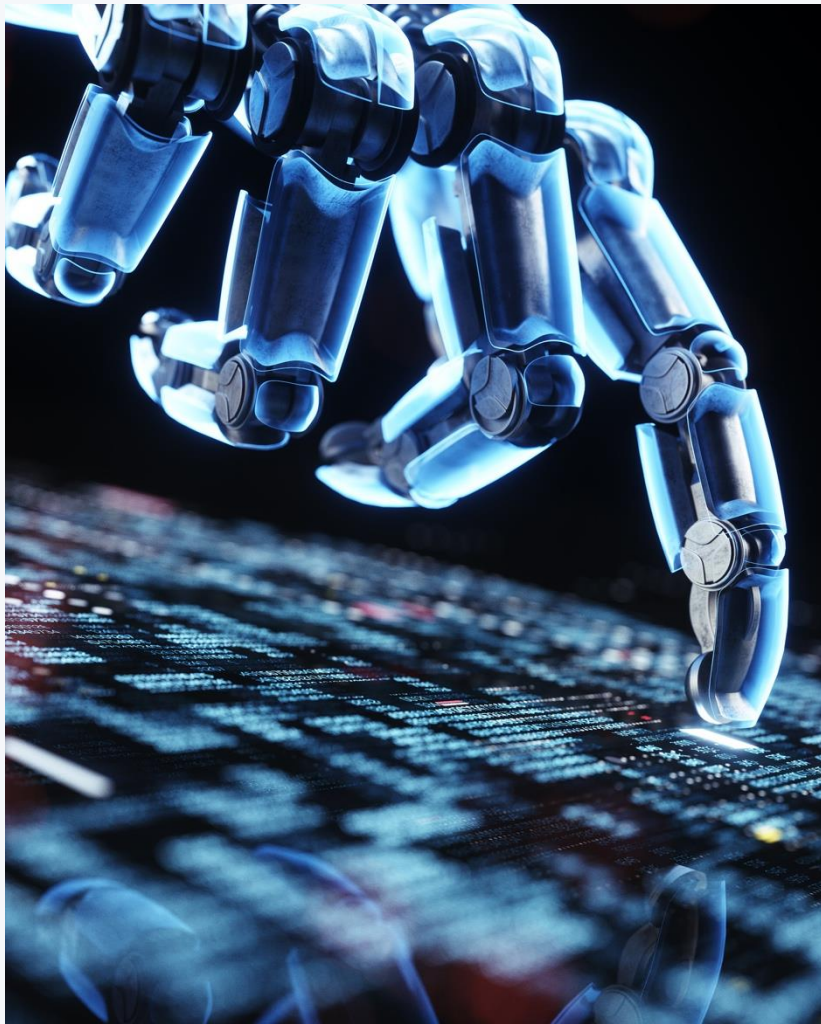
Wrong discount formula
implemented

Root cause

Product owner lacked
correct knowledge

Error

Human misunderstanding
while specifying the rule



1.3 Testing Principles

1.3 Testing Principles

Seven principles provide general guidance across contexts.



1.3

1. Presence, not absence

2. Exhaustive testing impossible

3. Early testing saves time

4. Defects cluster

5. Tests wear out

6. Context dependent

7. Absence-of-defects fallacy

1.3 Principles 1–2

Testing finds evidence of defects, but cannot test everything.



1. Testing shows the presence, not the absence of defects

- Finding no defects does not prove perfection.
- Testing reduces uncertainty; it does not prove correctness.



2. Exhaustive testing is impossible

- Trying every input, path, configuration, and usage pattern is unrealistic.
- Use prioritization, risk, and test techniques instead.



INTERACTIVE: Challenge question: why is “no bugs found” a dangerous release statement on its own?

1.3 Principles 3–4

1.3

Earlier feedback is cheaper; defects tend to concentrate.



3. Early testing saves time and money

- Defects found early do not propagate into later work products.
- Static reviews and early analysis help reduce total cost of quality.



4. Defects cluster together

- A small number of components often contain most defects or failures.
- Pareto rule: A small number of causes cause a large number of effects. Approximately 80% of defects may be found in approximately 20% of the components.

1.3 Principles 5–6

Old tests lose power; no single approach fits every project.



1.3



5. Tests wear out (Pesticide Paradox)

- Repeating the same tests eventually finds fewer new defects.
- Refresh tests and test data to keep them effective.



6. Testing is context dependent

- Safety-critical, banking, and game systems emphasize different risks and qualities.
- Choose techniques, coverage, and automation based on context.

1.3 Principle 7

Absence-of-defects fallacy: correctness alone is not enough.



1.3

**A system may be defect-free
against its specification ...**



**... but still fail user needs
or business goals.**



INTERACTIVE: Ask for a real-world example of software that “worked as built” but was still a poor product.

1.3 Activity: Principle diagnosis

Match the scenario to the most relevant principle.



1.3

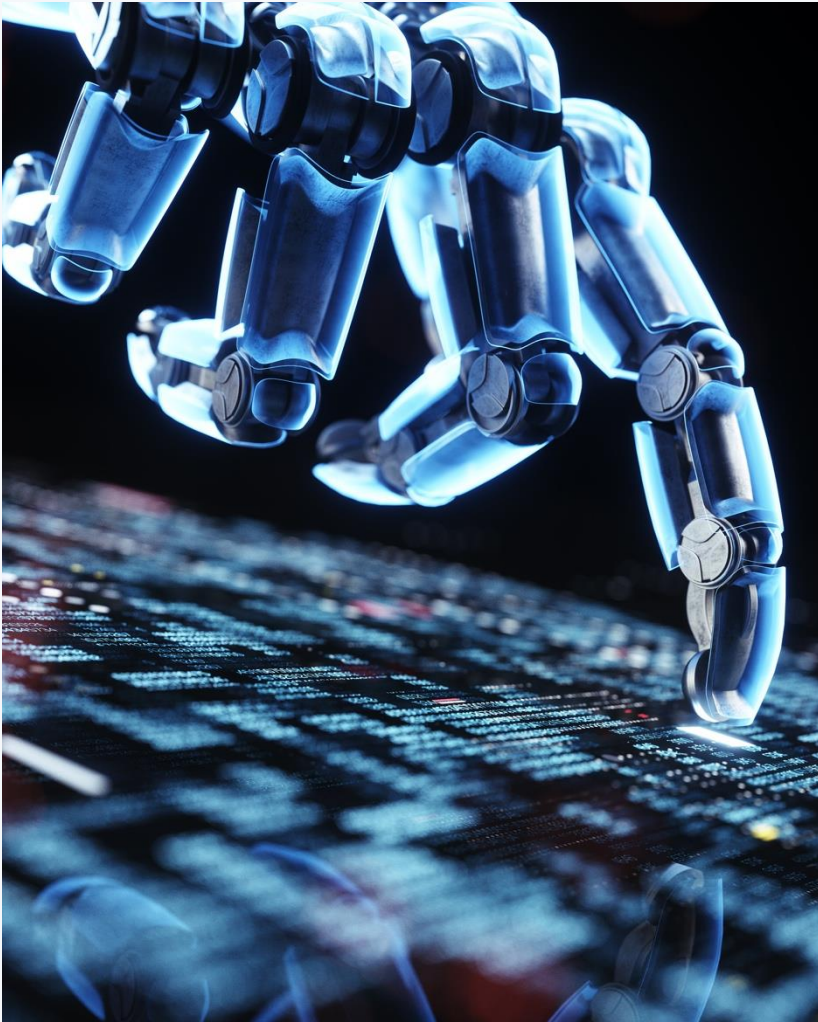
A. A repeated smoke suite finds no new issues for months.

B. A requirement defect found early prevents rework in coding.

C. Most production incidents come from the reporting module.

D. A fully tested feature still does not satisfy customer needs.

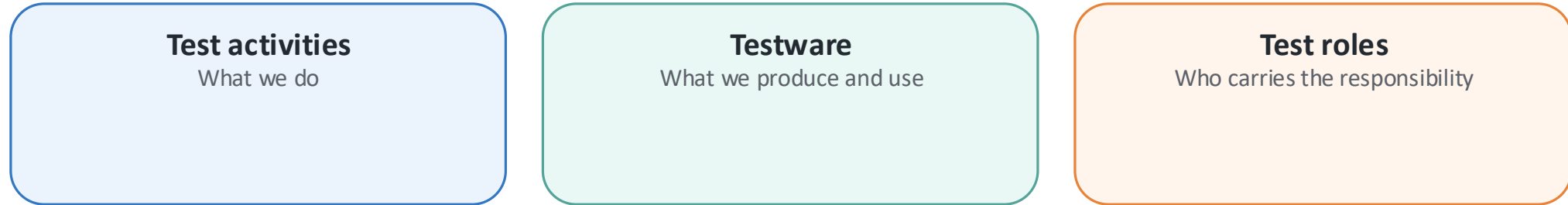
E. A test manager decides not to test every input combination.



1.4 Test Activities, Testware and Test Roles

1.4 Test Activities, Testware, and Test Roles

Testing has a process, produces work products, and involves distinct roles.



Key idea: testing is context dependent, but most projects still perform a recognizable sequence of activities and create related work products.

1.4.1 Test Activities and Tasks



1.4.1 Test planning + Test monitoring and control

The test process starts by deciding what to achieve and stays on track by monitoring progress.



Test planning

- Define test objectives.
- Select the test approach and techniques.
- Decide activities, tasks, schedule, and metrics.
- Planning is ongoing, not one-time only. (revised based on feedback from test monitoring & test control activities)



Test monitoring and control

- Compare actual progress with the plan.
- Use metrics and exit criteria to detect variance.
- Take corrective actions when testing deviates.

1.4.1 Test analysis + Test design

Analysis answers “what to test?” and design answers “how to test?”



Test basis

Test conditions

Coverage / priorities

Test cases



Test analysis

- Study the test basis to identify testable features.
- Define and prioritize test conditions and determine “what to test”
- Also helps detect defects in the test basis itself.



Test design

- Transform test conditions into test cases and related testware.
- Identify test data, environment needs, and coverage items.
- Answer “how to test”

1.4.1 Test implementation + test execution

Implementation prepares executable assets; execution runs them and records outcomes.



Test implementation

- Create or finalize test procedures, suites, scripts, and concrete test data. (Do we have everything we need to run the test)
- Prepare and verify the test environment.
- Arrange the execution schedule.



Test execution

- Run manual or automated tests.
- Compare actual and expected results.
- Log results, analyze anomalies, and report defects.
- Retest and run regression as needed.

1.4.1 Test completion

Testing ends by closing the loop and preserving value for future work.



1.4.1



Typical completion activities

- Check unresolved defects and create change requests or backlog items.
- Archive useful testware and hand it over where needed.
- Return the environment to an agreed state.
- Capture lessons learned and improvement actions.
- Create and communicate the test completion report to stakeholder.

1.4.2 Test Process in Context

1.4.2 Test process in context

The same process is shaped differently by the project environment.

Testing is not performed in isolation. Test activities are an integral part of the development processes carried out within an organization. Testing is also funded by stakeholders and its final goal is to help fulfill the stakeholders' business needs. Therefore, the way the testing is carried out will depend on a number of contextual factors including:

Stakeholders (needs, expectations, requirements, willingness to cooperate, etc.)

Team members (skills, knowledge, level of experience, availability, training needs, etc.)

Business domain (criticality of the test object, identified risks, market needs, specific legal regulations, etc.)

Technical factors (type of software, product architecture, technology used, etc.)

Project constraints (scope, time, budget, resources, etc.)

Organizational factors (organizational structure, existing policies, practices used, etc.)

Software development lifecycle (engineering practices, development methods, etc.)

Tools (availability, usability, compliance, etc.)

These factors will have an impact on many test-related issues, including: test strategy, test techniques used, degree of test automation, required level of coverage, level of detail of test documentation, reporting, etc.

1.4.3 Testware

1.4.3 Testware

Testing produces work products at every activity, not only test cases.



1.4.3

Activity	Example of Testware
Test Planning	test plan, risk register, entry/exit criteria
Test Monitoring and Test Control	Test progress reports, documentation of control directives, risk information
Test Analysis	test conditions, acceptance criteria
Test Design	test cases, coverage items, data/environment requirements
Test Implementation	procedures, scripts, suites, test data, environment elements
Test Execution	test logs, defect reports, status information
Test Completion	completion report, lessons learned, change requests

1.4.4 Traceability between the Test Basis and Testware

1.4.4 Traceability between test basis and testware

Traceability connects requirements, risks, tests, results, and defects.



Value of traceability

- supports coverage evaluation
- supports impact analysis when change occurs
- facilitates audits and reporting
- helps stakeholders understand status and residual risk

1.4.5 Roles in Testing

1.4.5 Roles in testing

The syllabus highlights two principal roles.



1.4.5



Test management role

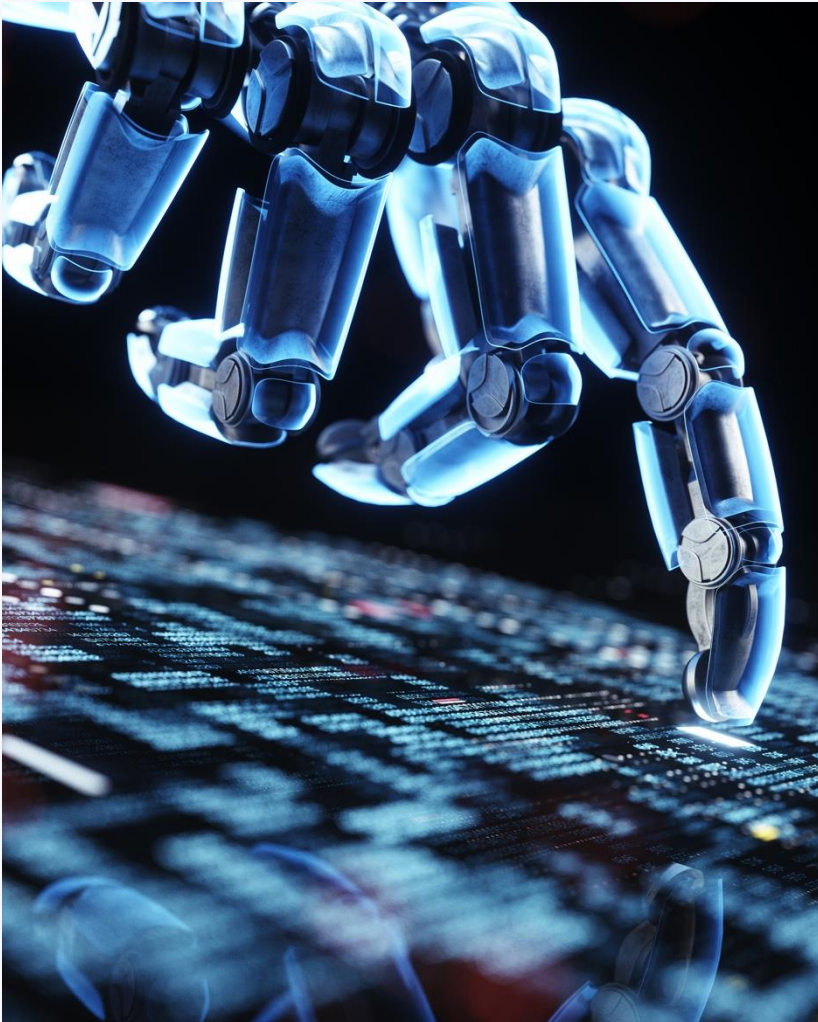
- Main focus: planning, monitoring/control, completion.
- Organizes the process, team, metrics, and coordination.
- May be a test manager, team lead, quality manager, or even the agile team.



Testing role

- Main focus: analysis, design, implementation, execution.
- Owns the technical / engineering side of testing.
- Can be performed by testers, developers, users, or other specialists depending on level.

Different people may take on these roles at different times. For example, the test management role can be performed by a team leader, by a test manager, by a development manager, etc. It is also possible for one person to take on the roles of testing and test management at the same time.



1.5 Essentials Skills and Good Practices in Testing

Essentials Skills and Good Practices in Testing

Skill is the **ability** to do something well that comes from one's knowledge, practice and aptitude.

Good testers should **possess some essential skills** to do their job well.

Good testers should be **effective team players** and should be able to perform testing on different levels of test independence.

1.5.1 Generic Skills Required for Testing

1.5.1 Generic skills required for testing

Effective testing needs both technical and interpersonal capability.



1.5.1

Testing knowledge

Thoroughness &
curiosity

Communication &
listening

Analytical / critical
thinking

Technical
knowledge

Domain knowledge

Because testers often bring bad news, constructive communication is a professional skill—not an optional extra.

1.5.2 Whole Team Approach

1.5.2 Whole team approach

Quality is a shared responsibility.



1.5.2

Developer

Tester

Business representative

Power of Three / Three Amigos
Discuss features together, clarify examples, agree acceptance tests.



Advantages

- better communication, stronger collaboration, broader skill use, shared responsibility for quality

Depending on the context, the **whole team approach may not always be appropriate**. For instance, in some situations, such as safety-critical, a high level of test independence may be needed.

1.5.3 Independence of Testing

1.5.3 Independence of testing

Independence can improve defect detection, but it also has trade-offs.



1.5.3



Benefits

- fresh perspective
- different biases and assumptions
- better chance of finding different defects
- can challenge stakeholder assumptions



Drawbacks

- possible isolation from the team
- communication friction or blame game
- developers may feel less responsible for quality
- risk of bottlenecks or missing context



INTERACTIVE: Debate: when is strong independence worth the cost? Consider safety-critical or regulated systems.

Chapter 1 recap and mini-checkpoint

Use this slide for review, discussion, or a short quiz.



Recap

1.1 what is testing?

1.1.2 testing vs debugging

1.2 Error / defect / failure / root cause

1.3 Seven principles

1.4 Test activities + testware + roles

1.5 Essentials skills + whole team + independence

Checkpoint questions

1. Why is testing more than execution?
2. Which principle explains focusing on a few high-defect modules?
3. What is the difference between QA and testing?
4. What does traceability help you evaluate?

Question?